XRationals.jl

Exact rational arithmetic with IEEE-like special values (NaN, Inf, -Inf), overflow-safe saturation, and lazy normalization for impressive throughput.

Types

Type	Alias	Backing	Overflow	Normalization
`XRational32`	`Qx32`	`Int32`	Saturates to Inf/NaN	Lazy (Int64 intermediate)
`XRational64`	`Qx64`	`Int64`	Saturates to Inf/NaN	Lazy (Int128 intermediate)
`XRational128`	`Qx128`	`Int128`	Saturates to Inf/NaN	Lazy (Int256 intermediate)
`XRational256`	`Qx256`	`Int256`	Saturates to Inf/NaN	Lazy (Int512 intermediate)
`XRational512`	`Qx512`	`Int512`	Saturates to Inf/NaN	Lazy (Int1024 intermediate)

Features

- Exact rational arithmetic with no floating-point rounding
- IEEE-like NaN, Inf, -Inf encoded in the same struct (`0//0`, `1//0`, `-1//0`)
- Overflow saturates to Inf/NaN instead of crashing (Qx types)
- Lazy GCD normalization: deferred until display, hashing, or conversion
- 3-13x faster than `Rational{Int}` for chained arithmetic
- Fused multiply-add (`fma`) with exact intermediate computation
- Cross-width narrowing (`Qx512 -> Qx256`, `Qx512 -> Qx128`, `Qx512 -> Qx64`, `Qx512 -> Qx32`, `Qx256 -> Qx128`, `Qx256 -> Qx64`, `Qx256 -> Qx32`, `Qx128 -> Qx64`, `Qx128 -> Qx32`, `Qx64 -> Qx32`) via best rational approximation
- Exact widening across exported extended types through constructor-first APIs (`Qx64(x::Qx32)`, `Qx128(x::Qx32)`, `Qx128(x::Qx64)`, `Qx256(x::Qx32)`, `Qx256(x::Qx64)`, `Qx256(x::Qx128)`, `Qx512(x::Qx32)`, `Qx512(x::Qx64)`, `Qx512(x::Qx128)`, `Qx512(x::Qx256)`), with `convert` and `widen` following the same width ladder
- Zero heap allocation: all arithmetic uses fixed-width integers (Int32/Int64/Int128/Int256/Int512/Int1024/Int2048)
- `typemin` rejection prevents silent negation overflow

Examples

See Use.jl for examples.

Benchmarks

The package includes a runnable benchmark harness at `test/Benchmark.jl`:

```
julia --project=. test/Benchmark.jl
```

It compares `Qx32`, `Qx64`, `Qx128`, `Qx256`, and `Qx512` against `Rational{Int32}`, `Rational{Int64}`, `Rational{Int128}`, `Rational{Int256}`, and `Rational{Int512}` using `Chairmarks`.

Representative local results from that harness:

Qx32 vs Rational{Int32}

Operation	Rational{Int32}	Qx32	Speedup
`a + b`	13 ns	2 ns	7.0x
`a * b`	8 ns	2 ns	4.2x
`a / b`	7 ns	2 ns	3.6x
`muladd(a,b,a)`	25 ns	3 ns	7.5x
`a+b+c+d`	66 ns	5 ns	14.3x
`a*b-c*d`	40 ns	4 ns	10.1x

Qx64 vs Rational{Int64}

Operation	Rational{Int64}	Qx64	Speedup
`a + b`	14 ns	3 ns	5.3x
`a * b`	9 ns	2 ns	4.0x
`a / b`	8 ns	3 ns	3.2x
`muladd(a,b,a)`	28 ns	6 ns	4.6x
`a+b+c+d`	72 ns	8 ns	9.4x
`a*b-c*d`	43 ns	5 ns	8.2x

Qx128 vs Rational{Int128}

Operation	Rational{Int128}	Qx128	Speedup
`a + b`	75 ns	7 ns	10.4x
`a * b`	65 ns	6 ns	10.8x
`a / b`	60 ns	7 ns	8.9x
`muladd(a,b,a)`	144 ns	12 ns	11.7x
`a+b+c+d`	269 ns	20 ns	13.2x
`a*b-c*d`	216 ns	17 ns	12.5x

Qx256 vs Rational{Int256}

Operation	Rational{Int256}	Qx256	Speedup
`a + b`	267 ns	23 ns	11.4x
`a * b`	230 ns	16 ns	14.3x
`a / b`	208 ns	19 ns	11.1x
`muladd(a,b,a)`	438 ns	38 ns	11.4x
`a+b+c+d`	996 ns	69 ns	14.5x
`a*b-c*d`	791 ns	53 ns	14.8x

Qx512 vs Rational{Int512}

Operation	Rational{Int512}	Qx512	Speedup
`a + b`	583 ns	93 ns	6.3x
`a * b`	461 ns	59 ns	7.9x
`a / b`	417 ns	64 ns	6.5x
`muladd(a,b,a)`	941 ns	156 ns	6.0x
`a+b+c+d`	2.1 us	310 ns	6.8x
`a*b-c*d`	1.7 us	222 ns	7.9x

`fma` is the main exception: the exported `Qx` types route finite fused multiply-add through a slower exact-or-canonical path before converting back to the fixed-width representation, while `Rational` just performs `muladd`. If you do not need that guarantee, `muladd` is the faster path.